

Guía completa — Bot de Telegram de NODO (de cabo a rabo)

Documento maestro. Explica qué es, cómo funciona por dentro (nodo por nodo) y cómo conectarlo, para que cualquier persona lo entienda completo y lo deje funcionando. No hace falta saber cómo se construyó.

Índice

1. Para qué sirve
2. Cómo funciona en una imagen
3. El flujo por dentro, nodo por nodo
4. Cómo conversa el bot (qué pregunta)
5. Configuración paso a paso (conectarlo)
6. Probar que quedó bien
7. Escalar para mucha gente
8. Solución de problemas
9. Privacidad y gobernanza
10. Datos del workflow

1. Para qué sirve

En la emergencia, la información de NODO (personas desaparecidas, refugios, hospitales, etc.) está en su plataforma web (nodoayuda.com). Mucha gente no entra a una web: usa **Telegram**.

Este bot deja que **cualquiera pregunte por Telegram** y reciba esa información al instante. Lee la **misma base de datos** que usa la web de NODO, en solo lectura.

Decisiones de diseño (por qué está hecho así):

- **Sin inteligencia artificial de pago (sin LLM):** clasifica lo que pide la gente con

palabras clave. Resultado: **costo casi cero**, aguanta muchísimas consultas.

- **Solo lectura:** el bot nunca escribe en la base de NODO. Cero riesgo de dañar datos.
- **Con buffer (Redis):** evita responder dos veces lo mismo y frena a quien escriba de más,

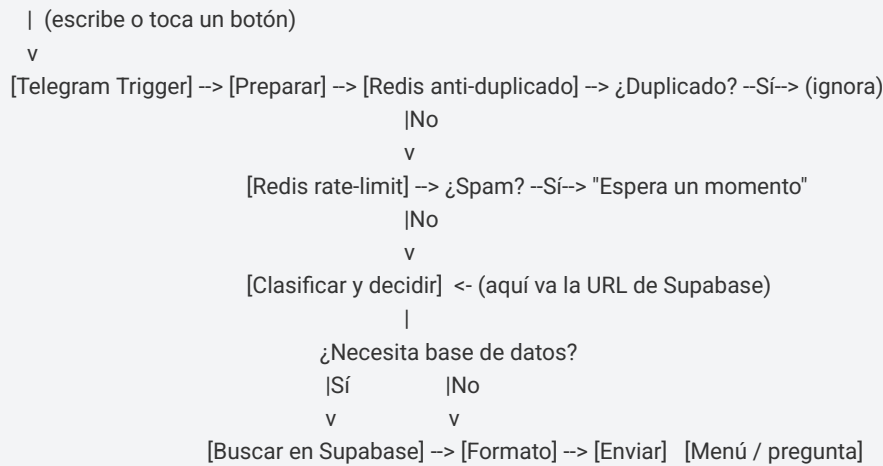
para que el sistema no se sature y **a todos se les responda.**

- **Escalable:** preparado para correr en ***queue mode*** (varios trabajadores en paralelo).

Las 7 cosas que responde: desaparecidos, refugios, hospitales, **sangre**, **centros de acopio**, campañas e información verificada.

2. Cómo funciona en una imagen

Persona en Telegram



En una frase: llega el mensaje -> se filtra spam/duplicados con Redis -> se entiende qué pide -> se busca en la base de NODO -> se le responde bonito por Telegram.

3. El flujo por dentro, nodo por nodo

El workflow se llama `NODO - Bot Telegram (avanzado)` y tiene 16 nodos (+ notas). Esto es lo que hace cada uno, en orden:

#	Nodo	Qué hace
1	Telegram Trigger	Es la "puerta de entrada". Se dispara cada vez que alguien le escribe al bot o toca un botón.
2	Preparar (Code)	Saca los datos del mensaje: quién escribe (chatId), el texto, si tocó un botón, y el id único del mensaje (update_id).
3	Redis: anti-duplicado	Cuenta cuántas veces se ha visto ese update_id (Telegram a veces reenvía). Guarda el dato 1 hora.
4	Leer duplicado (Code)	Lee ese contador.
5	¿Es duplicado? (IF)	Si el mensaje ya se procesó antes -> lo manda a "Ignorar" (no responde dos veces). Si es nuevo -> sigue.
6	Redis: rate-limit	Cuenta cuántos mensajes mandó esa persona en los últimos 8 segundos.
7	Leer rate (Code)	Lee ese contador.
8	¿Excede rate? (IF)	Si mandó más de 6 en 8s -> le dice "espera un momento" y no recarga el sistema. Si no -> sigue.
9	Clasificar y decidir (Code)	El cerebro . Normaliza el texto (quita acentos/mayúsculas), entiende si es / start , un botón o texto libre, decide la categoría (desaparecidos, refugios,

#	Nodo	Qué hace
		etc.) y arma la dirección exacta para consultar la base. Aquí se pone la URL de Supabase.
10	¿Necesita base de datos? (IF)	Si hay que buscar algo -> va a la base. Si solo hay que mostrar el menú o hacer una pregunta -> responde directo.
11	Buscar en Supabase (NODO) (HTTP)	Le pide los datos a la base de NODO por su API (con la anon key , solo lectura).
12	Fuzzy match y formato (Code)	Filtra por el nombre que pidieron (aunque tenga acentos distintos), ordena, y arma el mensaje de respuesta bonito.
13	Enviar resultados (Telegram)	Le manda la respuesta a la persona, con el menú para seguir buscando.
14	Enviar menú / mensaje (Telegram)	Manda la bienvenida, las preguntas ("escribeme el nombre...") o el "no te entendí".
15	Aviso espera (Telegram)	El mensaje de "espera" del anti-spam.
16	Ignorar duplicado (NoOp)	No hace nada: corta los mensajes repetidos.

*El "buffer de Redis" son los nodos 3-8: protegen el sistema (anti-duplicado + anti-spam). La **escala real** viene de correr n8n en **queue mode** (sección 7), que usa el **mismo Redis** como cola para repartir el trabajo entre varios trabajadores.*

4. Cómo conversa el bot

El usuario...	El bot responde...
/start, "hola", "menú", "ayuda"	Bienvenida + menú de 7 botones .
Toca Desaparecidos	*"Escribeme el nombre y apellido de la persona que buscas."* -> la persona escribe el nombre -> muestra coincidencias.
Toca Sangre	*"Escribe el tipo de sangre (ej: O+)."* -> escribe O+ -> muestra solicitudes de ese tipo.
Toca Refugios / Hospitales / Acopio / Campañas / Info	Responde directo con la lista.
"busco a María González" (texto libre)	Detecta búsqueda de persona -> coincidencias.
Un nombre solo: *"Pedro Pérez"*	Lo busca como persona.
"refugio", *"hospital"*, *"O+"*, *"acopio"*, *"campañas"*, *"qué está pasando"*	Detecta la categoría sola y responde.

El usuario...	El bot responde...
Algo que no entiende	*"No te entendí bien..."* + menú.
Muchos mensajes muy rápido	*"Estás enviando muchas consultas muy rápido. Dame un momento..."*

De dónde sale cada respuesta:

Categoría	Tabla de NODO	Muestra
Desaparecidos	persons	Nombre, estado, ciudad, última ubicación
Refugios	shelters	Nombre, dirección, ciudad, cupo
Hospitales	points_of_interest (hospital)	Nombre, ciudad, dirección
Sangre	points_of_interest (health_request)	Hospital, tipo de sangre, ubicación
Acopio	points_of_interest (collection_center)	Nombre, dirección, ciudad
Campañas	official_campaigns	Título, organización
Info verificada	verified_info	Título, contenido, fuente

5. Configuración paso a paso

Tiempo: 15–20 min. Necesitas: acceso a n8n, la URL + anon key de Supabase de NODO, el token del bot (@BotFather) y datos de un Redis (host/puerto/password).

Paso 1 — URL de Supabase (1 línea)

Abre el nodo "Clasificar y decidir". En la primera línea cambia:

```
const SUPABASE_BASE = 'https://YOUR-PROJECT.supabase.co';
```

por la URL real de NODO (sin barra al final). **Save**.

Paso 2 — Credencial de lectura (anon key)

Credentials -> New -> "Header Auth":

- Name: **apikey**
- Value: la **anon key** de NODO
- Nombre: **NODO Supabase anon** -> **Save**.

Luego, en el nodo "Buscar en Supabase (NODO)" -> *Authentication* -> **Generic Credential Type** -> **Header Auth** -> elige **NODO Supabase anon**.

Paso 3 — Bot de Telegram

En @BotFather -> /newbot -> copia el token. En n8n: Credentials -> New -> "Telegram API" -> pega el token -> nómbrala **NODO Telegram Bot**. Verifica que esté en los 4 nodos de Telegram (**Telegram Trigger**, **Aviso espera**, **Enviar resultados**, **Enviar menu / mensaje**).

Paso 4 — Redis (el buffer)

Credentials -> New -> "Redis" -> host, puerto (6379), password. Asígnala a "Redis: anti-duplicado" y "Redis: rate-limit". *(Usa el mismo Redis del queue mode si ya tienes uno.)*

Sin Redis configurado, esos 2 nodos fallan y el bot no responde.

Paso 5 — Activar

Pon el toggle en "Active". n8n registra el webhook del bot **solo**. [OK]

*[!] Activa **una sola** versión por bot (no la simple y la avanzada a la vez).*

6. Probar que quedó bien

En Telegram, con el bot:

- [] **/start** -> sale el menú de 7 botones.
- [] **Desaparecidos** -> pide nombre -> escribe uno real -> responde.
- [] **Refugios** -> lista (o "no hay" si la base está vacía).
- [] Escribe **"O+"** -> solicitudes de sangre.
- [] 8 mensajes rápidos -> en algún punto dice **"espera un momento"** (anti-spam OK).
- [] Reenvía el mismo mensaje -> no responde dos veces (anti-duplicado OK).

7. Escalar para mucha gente

El bot ya está listo. Para aguantar **picos de mucha gente**, n8n debe correr en **queue mode**: las consultas se encolan en Redis y varios ***workers*** las atienden en paralelo. **Esa cola es el buffer que hace que a todos se les responda** (en orden, sin que el sistema se caiga).

- Archivos: **deploy/docker-compose.queue.yml** + **deploy/README-deploy.md** (en la carpeta del proyecto).
- Levantar: **docker compose -f docker-compose.queue.yml --env-file .env up -d**
- Escalar: **--scale n8n-worker=3**
- **Techo real = Telegram**: 30 mensajes/seg por bot (los que **ENTRAN** no cuentan). La cola

amortigua los picos; el bot responde **1 mensaje por consulta**. Si se queda corto -> ***Paid Broadcasts*** en @BotFather (hasta 1000/seg).

8. Solución de problemas

Síntoma	Causa probable	Solución
El bot no responde	Workflow inactivo / token equivocado	Actívalo; verifica que la credencial Telegram sea el bot de NODO
Error en nodo Redis	Credencial Redis mal o host inalcanzable	Revisa host/puerto/password
Siempre "no encontré"	SUPABASE_BASE mal, apikey mal, o	Revisa Paso 1 y 2; prueba la URL REST

Síntoma	Causa probable	Solución
	base vacía	en el navegador
Error 401 en "Buscar en Supabase"	anon key mala o el header no se llama apikey	Revisa la credencial Header Auth (Name = apikey)
El botón se queda "cargando"	Falta answerCallbackQuery (cosmético)	No afecta la respuesta; se agrega en una v3

9. Privacidad y gobernanza

- **Privacidad:** la lectura ya es pública por diseño de NODO (RLS). Conviene revisar con el

admin qué campos sensibles (ubicación exacta) se muestran. El bot limita resultados y trae rate-limit anti-scraping.

- **Gobernanza:** este es un flujo externo que consume la API pública de NODO; no toca

el repo [NODOPlatform/Nodo](#). NODO tiene un *TODO* para un provider de Telegram in-repo ([src/assist/providers/telegram.ts](#)). Confirmar con el admin si quieren el bot externo (este) o el in-repo, para no duplicar esfuerzo.

10. Datos del workflow

- **Nombre:** [NODO - Bot Telegram \(avanzado\)](#)
- **n8n:** <https://flowfy-n8n.kzkkfj.easypanel.host/workflow/4WPhUyRR3l4Xfdzv>
- **Versión simple (respaldo):** [NODO - Bot Telegram \(consulta\)](#) — 4 categorías, sin Redis.
- **Tecnología:** n8n + Telegram Bot API + Supabase REST (anon key) + Redis. Sin LLM.